

NBoW Project Report

Nathaniel, Osborn

Popy, Halder

Dayvon, King

Introduction

A Neural Bag of Words (NBoW) model is the framework for teaching a computer to read text and guess whether the sentiment of the text is positive or negative by looking at which words appear, but ignoring the order they appear in. This model is necessary because one's feelings on social media can be easily misinterpreted; more importantly, sometimes people may send out indirect signs of distress or depression on public platforms. Depression is a detrimental issue that people can encounter at any age in their life. A model like NBoW can possibly help to identify and interpret if people are experiencing negative feelings.

Methodology

First, we load in our tools, then we lock the random number of generators to ensure that every time we run this code, the computer makes the same random guesses to start; it guarantees that our results are reproducible. 25,000 reviews (data) are downloaded and divided into three buckets; this helps keep integrity intact by eliminating the possibility that the model memorizes the answers. The training dataset (18,750 reviews) is like textbooks that the model studies. Validation dataset (6,250 reviews) can be thought of as the practice quiz we give the model while it learns, so that we can see if it is efficiently understanding the data. Lastly, the Test dataset is the full 25,000 reviews, which is like the final exam. The model never sees this until the very end. Next, we implement a basic tokenization, which will turn English text into a set of known integers for the model to understand. It creates a vocab dictionary like "movie" = 34, or "good" = 152. Chop sentences into lists of words. "I love IT" becomes ['i', 'love', 'it']. When building vocabulary for our model, we count every word. If it appears at least 5 times, it gets a unique ID number. Rare words are ignored to save memory. Special tokens are added, <unk> for words we don't know, and <pad>, which acts like packing peanuts to fill space. Then we define the `collate_batch` function that takes a handful of jagged sentences (some short, some long) and forces them into a perfect rectangle (a Tensor) so the math will work. Think of this process as an assembly line. We grab 512 reviews at a time, but because the reviews are different lengths, we can't stack them neatly, so

we use padding. By finding the longest review in the batch, we measure every other review against it and fill the empty spaces with the <pad> token. This allows us to have a perfect rectangular matrix of numbers. From there, we define the neural network logic, consisting of three layers. The first is the Embedding (nn.Embedding) layer, where we trade every word ID for a row of 300 floats. These numbers represent the 'meaning' of the word. Initially, these are random, but the model learns them over time. The second being the linear layer, acting as the judge, drawing a line to decide if the token falls on the Positive or Negative side. The third being The Mean Pooling (embedded.mean(dim=1)). Here, we take all the word vectors in a sentence and average them together; the words will lose individual semantic value, but the overall meaning of the sentence is clearer. Then we add the Training Loop, where the model is put through a cycle of guessing and correcting 10 times (epochs). First, the model reads a batch of reviews. Secondly, it compares its guess to the real label (0 or 1) using CrossEntropyLoss. Next, Backpropagation will calculate what the blame for the error. Then the Optimizer comes into effect by slightly nudging the numbers in the Embedding and Linear layers to reduce the error next time.

Challenges

One challenge we encountered was adding trustworthiness.

- Data quality & bias: The model learns from reviews or social media posts, but these texts can be biased, sarcastic, or misleading. If the training data isn't representative, the model predictions may not be reliable.
- Loss of nuance: NBoW approach ignores word order, which means subtle cues (like "not good" vs. "good") can be misinterpreted. This reduces reliability in capturing the true sentiment of a word/phrase.
- Human signals are complex: People express distress or sadness indirectly through metaphors, jokes, or cultural references; therefore, a model that only looks at word frequency may miss these signals, making its outputs less dependable.
- Generalization vs. memorization: Even with careful dataset splits, the model might overfit training examples into generalizing.
- Ethical responsibility: When dealing with sensitive topics like depression, false positives or negatives can have serious consequences. Trustworthiness isn't just technical; it's ensuring the model outputs can be responsibly used.

How does This Fits NBoW Project?

- Since NBoW aims to detect silent cries for help in text, trustworthiness becomes highly important. A model that mislabels emotions could either overlook someone in need or raise false alarms. That's why we framed "adding trustworthiness" as a challenge because it's about making sure the system is accurate, fair, and reliable enough to be trusted in real-world use.

Results and Conclusion

Output:

README.md:

```
7.81k/? [00:00<00:00, 617kB/s]
plain_text/train-00000-of-00001.parquet: 100%
21.0M/21.0M [00:01<00:00, 20.4MB/s]
plain_text/test-00000-of-00001.parquet: 100%
20.5M/20.5M [00:00<00:00, 38.2MB/s]
plain_text/unsupervised-00000-of-00001.p(...): 100%
42.0M/42.0M [00:01<00:00, 38.0MB/s]
Generating train split: 100%
25000/25000 [00:00<00:00, 89385.73 examples/s]
Generating test split: 100%
25000/25000 [00:00<00:00, 100832.80 examples/s]
Generating unsupervised split: 100%
50000/50000 [00:00<00:00, 124229.32 examples/s]
Train size: 18750
Valid size: 6250
Test size: 25000
Tokenizing training data...
```

```
Map: 100%
18750/18750 [00:06<00:00, 6799.16 examples/s]
Map: 100%
6250/6250 [00:00<00:00, 10583.90 examples/s]
Map: 100%
25000/25000 [00:01<00:00, 16241.20 examples/s]
Building vocabulary...
Vocabulary size: 38428
Model Parameters: 11,529,002
```

Starting Training...

Training: 100% 37/37 [00:33<00:00, 1.12it/s]

Epoch 1: Train Loss 0.683 (Acc: 0.601) | Valid Loss 0.669 (Acc: 0.663)

Training: 100% 37/37 [00:22<00:00, 1.65it/s]

Epoch 2: Train Loss 0.651 (Acc: 0.691) | Valid Loss 0.633 (Acc: 0.704)

Training: 100% 37/37 [00:23<00:00, 1.61it/s]

Epoch 3: Train Loss 0.603 (Acc: 0.738) | Valid Loss 0.584 (Acc: 0.745)

Training: 100% 37/37 [00:23<00:00, 1.58it/s]

Epoch 4: Train Loss 0.545 (Acc: 0.778) | Valid Loss 0.533 (Acc: 0.776)

Training: 100% 37/37 [00:23<00:00, 1.60it/s]

Epoch 5: Train Loss 0.487 (Acc: 0.811) | Valid Loss 0.486 (Acc: 0.803)

Training: 100% 37/37 [00:22<00:00, 1.63it/s]

Epoch 6: Train Loss 0.433 (Acc: 0.841) | Valid Loss 0.448 (Acc: 0.820)

Training: 100% 37/37 [00:22<00:00, 1.63it/s]

Epoch 7: Train Loss 0.387 (Acc: 0.864) | Valid Loss 0.418 (Acc: 0.832)

Training: 100% 37/37 [00:23<00:00, 1.57it/s]

Epoch 8: Train Loss 0.348 (Acc: 0.882) | Valid Loss 0.393 (Acc: 0.845)

Training: 100% 37/37 [00:22<00:00, 1.64it/s]

Epoch 9: Train Loss 0.314 (Acc: 0.896) | Valid Loss 0.373 (Acc: 0.854)

Training: 100% 37/37 [00:23<00:00, 1.57it/s]

Epoch 10: Train Loss 0.285 (Acc: 0.907) | Valid Loss 0.357 (Acc: 0.860)

--- Testing ---

This movie was terrible: ('NEGATIVE', 1.0)

I loved this film: ('POSITIVE', 0.9999972581863403)

Model weight output:

Analyzing model weights...

--- Top 20 MOST POSITIVE Words ---

great	(Score: 65.23)
excellent	(Score: 64.71)
superb	(Score: 60.20)
pleasantly	(Score: 57.87)
enjoyed	(Score: 57.83)
10/10	(Score: 57.42)
today.	(Score: 56.78)
hilarious	(Score: 56.28)
highly	(Score: 55.99)
favorite	(Score: 55.34)
fantastic	(Score: 55.20)
liked	(Score: 54.22)
refreshing	(Score: 53.80)
surprised	(Score: 52.58)
it!	(Score: 52.49)
gem.	(Score: 52.02)
pleasure	(Score: 51.59)
wonderful,	(Score: 51.38)
wonderful	(Score: 50.54)
true	(Score: 49.80)

--- Top 20 MOST NEGATIVE Words ---

worst	(Score: -75.53)
horrible	(Score: -66.20)
awful	(Score: -65.05)
bad	(Score: -63.35)
annoying	(Score: -63.11)
mess	(Score: -61.53)
redeeming	(Score: -61.53)
mess.	(Score: -61.51)
lame	(Score: -59.66)

ridiculous (Score: -58.84)
boring (Score: -57.03)
bad. (Score: -56.71)
stupid (Score: -56.68)
awful. (Score: -56.56)
wasted (Score: -56.41)
poorly (Score: -55.62)
script (Score: -54.84)
unless (Score: -54.77)
supposed (Score: -54.61)
terrible. (Score: -54.14)

Word: 'boring' Score: -57.03 -> NEGATIVE

Word: 'excellent' Score: 64.71 -> POSITIVE

Word: 'average' Score: 0.32 -> POSITIVE

Word: 'scary' Score: -9.63 -> NEGATIVE

Citations

1. Trevett, B. (2019, November). Neural Bag of Words [GitHub repository]. GitHub.
<https://github.com/bentrevett/pytorch-sentiment-analysis/blob/main/1%20-%20Neural%20Bag%20of%20Words.ipynb>
2. dair-ai. (2020). Emotion Dataset [Dataset]. Hugging Face.
<https://huggingface.co/datasets/dair-ai/emotion>

Contributions

Nate: 34% Code

Popy: 33% PowerPoint presentation

Dayvon: 33% Project Report creation

